

Exception Handling

1. ClassNotFoundException:

Description:

ClassNotFoundException is thrown when an application tries to load a class by its name during runtime, but the specified class is not available in the class path.

ClassNotFoundException is a checked exception in Java. It occurs when the Java Virtual Machine (JVM) tries to load a class at runtime but cannot find that class.

Task Description:

Task: Create a Java program that attempts to load a class named Student. Since the class does not exist, catch the ClassNotFoundException and display an appropriate message.

Step-by-Step Execution

Step 1

The main() method starts.

Step 2

Execution enters the try block.

Step 3

Class.forName("Student") asks JVM to load the Student class.

Step 4

JVM searches for the class.

Step 5

The class is not found.

Step 6

ClassNotFoundException is thrown.

Step 7

Control immediately moves to the catch block.

Step 8

The exception message is printed.

Step 9

Execution continues after the catch block.

Program

```
package exceptionhandling;
```

```
public class ClassNotFoundDemo {
```

```
    public static void main(String[] args) {
```

```
        try {
```

```
            Class.forName("Student");
```

```
            System.out.println("Class Loaded Successfully");
```

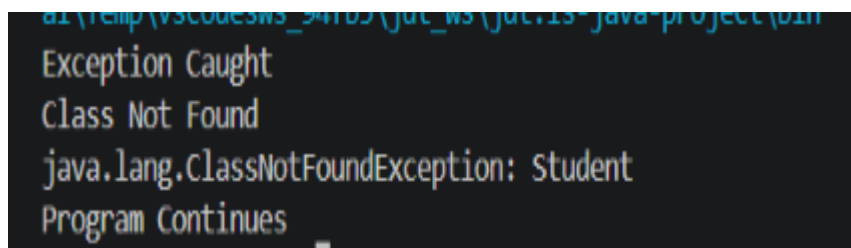
```
        }
```

```
        catch (ClassNotFoundException e) {
```

```
        System.out.println("Exception Caught");  
        System.out.println("Class Not Found");  
        System.out.println(e);  
    }
```

```
        System.out.println("Program Continues");  
    }  
}
```

Output:

A screenshot of a terminal window with a dark background. The output text is as follows:
Exception Caught
Class Not Found
java.lang.ClassNotFoundException: Student
Program Continues
The text is displayed in a monospaced font with some color coding: 'Exception Caught' and 'Program Continues' are in white, while 'Class Not Found' and the full exception message are in a light blue/cyan color.

```
ai/temp/vscodeWS_94703/jdt_ws/jdt.ls-java-project/bin  
Exception Caught  
Class Not Found  
java.lang.ClassNotFoundException: Student  
Program Continues
```

ArrayIndexOutOfBoundsException

Introduction

ArrayIndexOutOfBoundsException is a runtime (unchecked) exception in Java. It occurs when a program tries to access an array element using an index that is outside the valid range.

For an array of size n :

- First index = 0
- Last index = $n-1$

Accessing an index less than 0 or greater than $n-1$ throws this exception.

Definition

ArrayIndexOutOfBoundsException is thrown when an invalid index is used to access an array.

Task Description

Task: Create a Java program with an array of 5 elements. Attempt to access the 6th element (index 5). Handle the exception and display a proper message.

Example Program

```
package exceptionhandling;

public class ArrayIndexDemo {

    public static void main(String[] args) {

        int arr[] = {10,20,30,40,50};

        try {
```

```
        System.out.println("Element at index 2 : " + arr[2]);
        System.out.println("Element at index 5 : " + arr[5]);
    }
    catch(ArrayIndexOutOfBoundsException e){
        System.out.println("Exception Caught");
        System.out.println("Invalid Array Index");
        System.out.println(e);
    }
    System.out.println("Program Continues");
}
}
```

Algorithm

1. Start.
2. Create an integer array with five elements.
3. Enter the try block.
4. Access a valid index.
5. Attempt to access an invalid index.
6. JVM throws `ArrayIndexOutOfBoundsException`.
7. Catch the exception.
8. Display an error message.
9. Continue execution.
10. Stop.

Output:

```
Element at index 2 : 30  
Exception Caught  
Invalid Array Index  
java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 5  
Program Continues  
PS C:\Users\ketha>
```

3. **ClassCastException**

Introduction

ClassCastException is an unchecked (runtime) exception. It occurs when one object is incorrectly cast to another incompatible class.

For example, if an Integer object is cast to a String, Java throws ClassCastException.

Package: java.lang

No import statement is required.

Definition

ClassCastException is thrown when an object is cast to a class of which it is not an instance.

Task Description

Create a Java program that stores an Integer object inside an Object reference and then incorrectly casts it to String. Handle the exception using try-catch.

Program:

```
package exceptionhandling;
```

```
public class ClassCastDemo {
```

```
    public static void main(String[] args) {
```

```
        Object obj = Integer.valueOf(100);
```

```
        try {
```

```
            String text = (String) obj;
```

```
            System.out.println(text);
```

```
        }
```

```
        catch (ClassCastException e) {
```

```
            System.out.println("Exception Caught");
```

```
            System.out.println("Invalid Type Casting");
```

```
            System.out.println(e);
```

```
    }  
  
    System.out.println("Program Continues");  
}  
}
```

Algorithm

1. Start.
2. Create an Integer object.
3. Store it in an Object reference.
4. Enter the try block.
5. Attempt to cast it to String.
6. JVM throws ClassCastException.
7. Catch the exception.
8. Display the error message.
9. Continue execution.
10. Stop.

Output:

Exception Caught

Invalid Type Casting

java.lang.ClassCastException:

class java.lang.Integer cannot be cast to class java.lang.String

Program Continues

4. FileNotFoundException

Definition

FileNotFoundException is thrown when an attempt to open a file fails because the file does not exist or cannot be accessed.

Task Description

Create a Java program that attempts to open a file named **student.txt**.

Since the file does not exist, Java throws a FileNotFoundException.

Program:

```
package exceptionhandling;

import java.io.FileInputStream;
import java.io.FileNotFoundException;

public class FileNotFoundExceptionDemo {

    public static void main(String[] args) {
```

```
try {  
    FileInputStream file = new FileInputStream("student.txt");  
    System.out.println("File Opened Successfully");  
}  
catch (FileNotFoundException e) {  
    System.out.println("Exception Caught");  
    System.out.println("File Not Found");  
    System.out.println(e);  
}  
System.out.println("Program Continues");  
}  
}
```

Algorithm

1. Start.
2. Enter the try block.
3. Attempt to open student.txt.
4. JVM searches for the file.
5. If the file exists, open it.
6. Otherwise throw FileNotFoundException.
7. Move to the catch block.
8. Display the exception message.

9. Continue program execution.
 10. Stop.
-

Step-by-Step Execution

Step 1

Program starts.

Step 2

Execution enters the try block.

Step 3

FileInputStream file =
new FileInputStream("student.txt");
Java searches for student.txt.

Step 4

File is not found.

Step 5

JVM throws FileNotFoundException.

Step 6

Control moves to the catch block.

Step 7

Error message is displayed.

Step 8

Program continues normally.

Output:

Exception Caught

File Not Found

java.io.FileNotFoundException: student.txt

Program Continues

5. IOException

IOException is a Java exception that occurs when an input or output operation fails or is interrupted. It is commonly associated with files, streams, keyboard input, network input/output, and other data-transfer operations.

IOException is a checked exception, so Java normally requires it to be handled using try-catch or declared using throws.

2. Objective

To understand and demonstrate how an IOException can be handled in Java using the try-catch mechanism and display a suitable message when an input/output error occurs.

Question

Write a Java program to demonstrate IOException by attempting to open a file named student.txt. Handle the exception using a try-catch block and display an appropriate error message.

Java Program

```
package exceptionhandling;

import java.io.FileReader;
import java.io.IOException;

public class IOExceptionDemo {

    public static void main(String[] args) {

        try {

            FileReader file = new FileReader("student.txt");
            System.out.println("File Opened Successfully");

            file.close();

        } catch (IOException e) {

            System.out.println("Exception Caught");

            System.out.println("Input/Output Error");

            System.out.println(e);

        }

        System.out.println("Program Continues");
```

```
}  
  
}
```

Step-by-Step Execution

1. Start the program.
2. Enter the try block.
3. Attempt to open student.txt.
4. If the file exists, open and close it successfully.
5. If the file does not exist, IOException is thrown.
6. Control moves to the catch block.
7. Display the error message.
8. Print "Program Continues".
9. End the program.

Algorithm

1. Start.
2. Import FileReader and IOException.
3. Enter the try block.
4. Open student.txt using FileReader.
5. Print a success message if the file opens.
6. Close the file.
7. If an I/O error occurs, catch IOException.
8. Display the exception message.
9. Print "Program Continues".
10. Stop.

Output

Case 1: student.txt does not exist

Exception Caught

Input/Output Error

java.io.FileNotFoundException: student.txt (The system cannot find the file specified)

Program Continues

Case 2: student.txt exists

File Opened Successfully

Program Continues

6. NegativeArraySizeException

NegativeArraySizeException is a Java runtime exception that occurs when a program tries to create an array with a negative size. For example, `new int[-5]` is invalid because an array can have zero or a positive number of elements, but it cannot have a negative number of elements.

Objective

To understand and demonstrate how Java handles an attempt to create an array using a negative size, and to handle the NegativeArraySizeException using try-catch.

Task Description

Write a Java program that attempts to create an integer array with size -5. Since the array size is negative, Java throws `NegativeArraySizeException`. The exception must be caught and a suitable message should be displayed.

Step-wise Execution

1. Start the program.
2. Enter the `main()` method.
3. Set the array size variable to -5.
4. Try to create an integer array using `new int[size]`.
5. Java detects that the requested array size is negative and throws `NegativeArraySizeException`.
6. Control moves to the catch block.
7. The program prints: "Exception: Array size cannot be negative".
8. The program ends normally after handling the exception.

Algorithm

1. START
2. Set `size = -5`.
3. Begin try block.
4. Create an integer array of size `size`.

5. If the size is negative, catch `NegativeArraySizeException`.
6. Display the exception message.
7. STOP

Program Code

```
class NegativeArrayDemo {  
    public static void main (String[] args) {  
        try (  
            int size = -5;  
            int numbers = new int[size];  
            System.out.println (numbers.length);  
        } catch (NegativeArraySizeException e) {  
            System.out.println("Exception: Array size cannot be  
            negative");  
        }  
    }  
}
```

Code Explanation

. class `NegativeArrayDemo` - Defines the Java class

- . public static void main(String[] args) - The starting point of program execution.
- . try - Contains code that may generate an exception.
- . int size = -5; - Stores a negative array size.
- . int[] numbers = new int[size]; - Attempts to create an integer array. Because size is -5, Java throws NegativeArraySizeException.
- . System.out.println(numbers.length); - This statement is not reached because the exception occurs on the previous line.
- . catch (NegativeArraySizeException e) - Catches the specific exception.
- . System.out.println(...) - Displays a clear message explaining the problem.

Output:

Program Output

Exception: Array size cannot. be negative

NullPointerException

What is NullPointerException?

NullPointerException (NPE) is a runtime exception in Java that occurs when a program tries to use a reference variable that currently contains null. In this program, `name` is assigned null, so calling `name.length()` tries to access a method through a null reference. The exception is caught by the catch block and a friendly message is displayed.

Objective

To understand NullPointerException in Java and demonstrate how to handle it using try-catch exception handling.

Task Description

Write a Java program in which a String reference is assigned the value null. Attempt to access its length, observe the NullPointerException, and handle the exception using a catch block so that the program terminates normally with an appropriate message.

Step-wise Execution

1. Start the program.
2. Enter the `main()` method.
3. Create the String reference variable `name` and assign null to it.
4. Execute `name.length()`. Since `name` does not refer to a String object, Java throws NullPointerException.

5. Control moves from the try block to the matching catch block.
6. The catch block prints: "Exception: Object contains null value".
7. The program ends normally.

Algorithm

START

Declare a String variable name and assign null.

Enter the try block.

Call name.length().

If name is null, NullPointerException occurs.

Catch NullPointerException.

Display the exception-handling message.

Stop

Program Code

```
class NullPointerExceptionDemo {  
    public static void main(String[] args) {  
        try {  
            String name = null;  
            System.out.println(name.length());  
        } catch (NullPointerException e) {  
            System.out.println("Exception: Object contains null value");  
        }  
    }  
}
```

```
}  
}  
}
```

Code Explanation

`class NullPointerException` - Defines the class named `NullPointerException`.

`public static void main(String[] args)` - The main method is the starting point of the Java program.

`try { ... }` - Contains statements that may produce an exception.

`String name = null;` - Creates a `String` reference named `name` but it does not refer to any `String` object.

`name.length();` - Attempts to call `length()` through `name`. Because `name` is `null`, Java throws `NullPointerException`.

`catch (NullPointerException e)` - Catches the specific `NullPointerException` so the program can handle it.

`System.out.println( ... )` - Displays a meaningful message instead of allowing the exception to terminate

the program unexpectedly.

Output

Exception: Object contains null value

NumberFormatException

What is NumberFormatException?

NumberFormatException is a runtime exception in Java that occurs when a program tries to convert a String into a numeric value, but the String does not contain a valid number. For example, `Integer.parseInt("ABC")` cannot convert "ABC" into an integer, so Java throws NumberFormatException.

Objective

To understand and handle NumberFormatException in Java by converting a String into an integer using `Integer.parseInt()` and handling the invalid input with try-catch.

Task Description

Write a Java program that attempts to convert the String value "ABC" into an integer. Since "ABC" is not a valid numeric string, NumberFormatException is generated. Handle the exception using a catch block and display a meaningful error message.

Step-wise Execution

1. Start the Java program.
2. Declare a String variable value and assign it the value "ABC".
3. Use `Integer.parseInt(value)` to convert the String into an integer.
4. Java checks whether the String represents a valid integer.
5. Because "ABC" is not a valid integer, NumberFormatException is thrown.
6. Program control moves from the try block to the catch block.

7. The catch block displays: "Exception: Invalid number format".
8. The program ends normally after handling the exception.

Algorithm

- 1.Start
- 2.Set value = "ABC"
- 3.Try to convert value to integer using Integer.parseInt(value)
- 4.If conversion is successful, display the integer
- 5.If NumberFormatException occurs, catch it
- 6.Display "Exception: Invalid number format"
- 7.Stop

Code

```
class NumberFormatDemo {  
    public static void main(String[] args) {  
        try {  
            String value = "ABC";  
            int number = Integer.parseInt(value);  
            System.out.println(number);  
        } catch (NumberFormatException e) {  
            System.out.println("Exception: Invalid number format");  
        }  
    }  
}
```

Code Explanation

- . try: Contains the statements that may generate an exception.
- . String value = "ABC" ;; Stores a non-numeric string.
- . Integer.parseInt(value): Attempts to convert the String into an int.
- . NumberFormatException: Thrown because "ABC" cannot be converted into an integer.
- . catch (NumberFormatException e): Catches and handles the exception.
- . System.out.println(...): Displays a user-friendly error message.

Output

Exception: Invalid number format.

Security Exception

1. Security Exception Means

SecurityException is a Java runtime exception that occurs when a program tries to perform an operation that is not permitted by the security rules or access restrictions. In this example, the exception is deliberately generated using throw to demonstrate how it can be caught and handled.

2. Objective

To understand the `SecurityException` in Java and demonstrate how to generate, catch, and handle a `SecurityException` using try-catch.

3. Task Description

Write a Java program named `SecurityExceptionDemo` that throws a `SecurityException` with the message "Access denied" and catches it using a catch block. The program should display an appropriate message when the exception is handled.

4. Step-by-Step Execution

1. The program execution starts from the `main()` method.
2. A try block is entered.
3. The statement `throw new SecurityException("Access denied")` explicitly creates and throws a `SecurityException`.
4. Normal execution inside the try block stops immediately.
5. The matching catch (`SecurityException e`) block receives the exception.
6. The catch block executes `System.out.println()` and displays "Exception: Security access denied".
7. The program terminates normally after handling the exception.

5. Algorithm

Step 1: Start.

Step 2: Enter the `main()` method.

Step 3: Begin the try block.

Step 4: Throw a `SecurityException` with the message "Access denied".

Step 5: Catch the `SecurityException`.

Step 6: Display "Exception: Security access denied".

Step 7: Stop.

8. Code Explanation

`class SecurityExceptionDemo` - Defines a Java class named `SecurityExceptionDemo`.

`public static void main(String[] args)` - The main method is the starting point of program execution.

`try` - Contains code that may generate an exception.

`throw new SecurityException("Access denied")` - Creates a `SecurityException` object and explicitly throws it.

`catch (SecurityException e)` - Catches the `SecurityException` and stores it in the reference variable `e`.

`System.out.println("Exception: Security access denied")` - Displays the exception-handling message on the console.

Program:

```
class SecurityExceptionDemo {  
    public static void main(String[] args) {
```

```
try {  
    throw new SecurityException("Access denied");  
} catch (SecurityException e) {  
    System.out.println("Exception: Security access denied");  
}  
}  
}
```

Java Program Output:

Exception: Security access denied.

StringIndexOutOfBoundsException

1. What is StringIndexOutOfBoundsException?

StringIndexOutOfBoundsException occurs when a program tries to access a character in a String using an invalid index. Java String indexes start from 0 and go up to length - 1. In this program, the string "Java" has 4 characters, so valid indexes are 0, 1, 2, and 3. Trying to access index 10 causes the exception. The exception is caught and a meaningful message is displayed instead of terminating the program.

Note: This is different from NumberFormatException.

NumberFormatException occurs when a String

cannot be converted into a valid number, such as `Integer.parseInt("ABC")`.

2. Objective / Aim

To write and execute a Java program that demonstrates `StringIndexOutOfBoundsException` by accessing an invalid String index and handling the exception using try-catch.

3. Task Description

Create a Java class named `StringIndexDemo`. Store the String "Java" in a variable, attempt to access character index 10 using `charAt()`, catch the resulting `StringIndexOutOfBoundsException`, and display an appropriate exception message.

4. Step-wise Execution

1. Start the program.
2. Create a String variable named `text` and assign it the value "Java".
3. The String contains 4 characters, so its valid indexes are 0 to 3.
4. Call `text.charAt(10)` to access the character at index 10.
5. Index 10 is invalid because it is outside the valid range 0 to 3.
6. Java throws `StringIndexOutOfBoundsException`.
7. The catch block catches the exception.
8. Print "Exception: String index is out of bounds".

9. End the program.

5. Algorithm

1. START

2. Declare String text = "Java".

3. Try to execute text.charAt(10).

4. If the index is invalid, an exception is generated.

5. Catch StringIndexOutOfBoundsException.

6. Display the exception message.

7. STOP

8. Code Explanation

. class StringIndexDemo - Defines the Java class.

. main() - The execution of the program begins here.

. try - Contains code that may generate an exception.

. String text = "Java"; - Creates a String containing four characters.

. text.charAt(10) - Attempts to access the character at index 10. This is invalid because the maximum valid index is 3.

. catch (StringIndexOutOfBoundsException e) - Catches the exception generated by the invalid String index.

. System.out.println(...) - Displays a user-friendly message explaining the error.

Code:

```
class StringIndexDemo {  
    public static void main(String[] args) {  
        try {  
            String text = "Java";  
            System.out.println(text.charAt(10));  
        } catch (StringIndexOutOfBoundsException e) {  
            System.out.println("Exception: String index is out of bounds");  
        }  
    }  
}
```

Output:

Exception: String index is out of bounds.